# M5Stack Chain IMU UART 控制协议

## 1. 基本信息

| 项目 | 说明 |
| --- | --- |
| 通信方式 | UART / 串口异步通信 |
| 设备 | M5Stack Chain-IMU |
| 波特率 | 115200 bps |
| 数据位 | 8 bit |
| 停止位 | 1 bit |
| 校验位 | None |
| 流控 | 无 |
| 固件版本 | `0x01` |

## 2. 数据帧格式

### 2.1 请求帧格式

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Header | 2 byte | 固定为 `0xAA 0x55` |
| Length_low | 1 byte | Length 低字节，低字节在前 |
| Length_high | 1 byte | Length 高字节 |
| Index | 1 byte | 设备在 Chain 链路中的位置；普通命令通常使用设备 Index |
| Cmd | 1 byte | 请求指令码 |
| Req Data | 变长 | 请求参数；无请求参数时长度为 `0` |
| CRC | 1 byte | 见“3. 校验说明” |
| Tail | 2 byte | 固定为 `0x55 0xAA` |

```
Request Frame:
  AA 55 <Length_low> <Length_high> <Index> <Cmd> <Req Data...> <CRC> 55 AA
```

## 2.2 应答帧格式

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Header | 2 byte | 固定为 `0xAA 0x55` |
| Length_low | 1 byte | Length 低字节，低字节在前 |
| Length_high | 1 byte | Length 高字节 |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 应答指令码；普通命令与请求 Cmd 一致 |
| Resp Data | 变长 | 应答数据；字段结构由对应 Cmd 定义 |
| CRC | 1 byte | 见“3. 校验说明” |
| Tail | 2 byte | 固定为 `0x55 0xAA` |

```
Response Frame:
  AA 55 <Length_low> <Length_high> <Index> <Cmd> <Resp Data...> <CRC> 55 AA
```

## 2.3 Length 约定

| 项目 | 规则 |
| --- | --- |
| Length 字段 | 两字节 `Length_low`、`Length_high`，低字节在前 |
| Length 计算 | `Length = 3 + Req/Resp Data 长度` |
| Length 统计范围 | 从 `Index` 开始，到 `CRC` 结束；包括 `Index`、`Cmd`、Data 和 `CRC`，不包括 Header、Length 和 Tail |
| 无 Data 帧 | `Length = 0x0003` |
| 1 byte Data 帧 | `Length = 0x0004` |
| 2 byte Data 帧 | `Length = 0x0005` |
| 最大数据长度 | 由固件接收缓冲区和对应指令实现共同限制 |

## 2.4 特殊帧约定

| Cmd | 类型 | 说明 |
| --- | --- | --- |
| `0xE0` | IMU 事件上报 | 设备自动上报，主机不发送请求帧 |
| `0xFC` | 枚举请求 / 链路通知 | Chain 内部特殊帧，不作为普通命令处理；按当前固件链路逻辑转发 |
| `0xFD` | 心跳帧 | Chain 设备之间使用；心跳请求不需要主机响应 |

| Cmd | 类型 | 说明 |
| --- | --- | --- |
| `0xFE` | 枚举帧 | 用于统计 Chain 链路中的设备数量 |

# 3. 校验说明

| 项目 | 规则 |
| --- | --- |
| 校验字段 | CRC |
| 校验范围 | 从 `Index` 开始，累加到 Data 最后一个字节；不包括 Header、Length、CRC 和 Tail |
| 校验算法 | 所有参与校验的字节按无符号 8 bit 累加，取低 8 位 |
| 空 Data | `CRC = (Index + Cmd) & 0xFF` |
| 校验失败 | 固件丢弃该帧，不执行对应命令 |

```
CRC = (Index + Cmd + Data[0] + ... + Data[n]) & 0xFF
```

固件发送普通命令应答时，CRC 由 `Index`、`Cmd` 和 `Resp Data` 计算。Chain 链路转发帧会根据当前固件的 Index 变化同步调整 CRC；转发帧和特殊帧不改变其原有帧结构。

## 3.1 CRC 示例程序

```c
#include <stdint.h>

uint8_t chain_calc_crc(const uint8_t *buffer, uint16_t size)
{
    uint8_t crc = 0;

    for (uint16_t i = 0; i < size; i++) {
        crc += buffer[i];
    }

    return crc;
}
```

调用示例：

```c
// buffer 指向 Index，size 包含 Index、Cmd 和 Data，不包含 CRC
uint8_t crc = chain_calc_crc(buffer, data_size);
```

# 4. 指令快速总览

本节列出全部 Cmd。Length 按当前固件实际帧格式计算。

| Cmd | 指令名称 | 请求 Length | 应答 / 上报 Length | Req Data | Resp Data | 详情 |
|---|---|---|---|---|---|---|
| `0x13` | I2C 指定地址读 | `0x0008` | `0x0004 + length` | `I2c_addr`、`Reg_addr_length`、`Reg_addr`、`length` | `Operation_Status`、Data | 5.1 |
| `0x14` | I2C 指定地址写 | `0x0008 + length` | `0x0004` | `I2c_addr`、`Reg_addr_length`、`Reg_addr`、`length`、Data | `Operation_Status` | 5.2 |
| `0x20` | 设置 RGB 值 | `0x0005 + Num * 3` | `0x0004` | `Index`、`Num`、RGB 值 | `Operation_Status` | 5.3 |
| `0x21` | 获取 RGB 值 | `0x0005` | `0x0004 + Num * 3` | `Index`、`Num` | `Operation_Status`、RGB 值 | 5.4 |
| `0x22` | 设置 RGB 灯亮度 | `0x0005` | `0x0004` | `Light`、`Save_to_flash` | `Operation_Status` | 5.5 |
| `0x23` | 获取 RGB 灯亮度 | `0x0003` | `0x0004` | 无 | `Light` | 5.6 |
| `0x30` | 读取加速度数据 | `0x0003` | `0x0009` | 无 | `Accel_X/Y/Z` | 5.7 |
| `0x31` | 读取角速度数据 | `0x0003` | `0x0009` | 无 | `Gyro_X/Y/Z` | 5.8 |
| `0x32` | 读取加速度原始值 | `0x0003` | `0x0009` | 无 | `Accel_X/Y/Z` 原始值 | 5.9 |
| `0x33` | 读取角速度原始值 | `0x0003` | `0x0009` | 无 | `Gyro_X/Y/Z` 原始值 | 5.10 |
| `0x34` | 读取温度 | `0x0003` | `0x0005` | 无 | `Temperature_C_x100` | 5.11 |
| `0x35` | 设置加速度量程 | `0x0004` | `0x0004` | `Accel_Range` | `Operation_Status` | 5.12 |
| `0x36` | 读取加速度量程 | `0x0003` | `0x0004` | 无 | `Accel_Range` | 5.13 |
| `0x37` | 设置陀螺仪量程 | `0x0004` | `0x0004` | `Gyro_Range` | `Operation_Status` | 5.14 |
| `0x38` | 读取陀螺仪量程 | `0x0003` | `0x0004` | 无 | `Gyro_Range` | 5.15 |

| Cmd | 指令名称 | 请求 Length | 应答 / 上报 Length | Req Data | Resp Data | 详情 |
|---|---|---|---|---|---|---|
| 0x39 | 设置加速度输出频率 | 0x0004 | 0x0004 | Accel_ODR | Operation_Status | 5.16 |
| 0x3A | 读取加速度输出频率 | 0x0003 | 0x0004 | 无 | Accel_ODR | 5.17 |
| 0x3B | 设置陀螺仪输出频率 | 0x0004 | 0x0004 | Gyro_ODR | Operation_Status | 5.18 |
| 0x3C | 读取陀螺仪输出频率 | 0x0003 | 0x0004 | 无 | Gyro_ODR | 5.19 |
| 0xE0 | IMU 事件触发 | 无 | 0x0005 | 无；设备自动上报 | INT_Status、Report_Code | 5.20 |
| 0xE1 | 设置 IMU 事件触发开关 | 0x0004 | 0x0004 | Mode | Operation_Status | 5.21 |
| 0xE2 | 读取 IMU 事件触发开关 | 0x0003 | 0x0004 | 无 | Mode | 5.22 |
| 0xE3 | 设置 IMU 六轴阈值 | 0x0005 | 0x0004 | Accel_Threshold、Gyro_Threshold | Operation_Status | 5.23 |
| 0xE4 | 读取 IMU 六轴阈值 | 0x0003 | 0x0005 | 无 | Accel_Threshold、Gyro_Threshold | 5.24 |
| 0xE5 | 设置温度阈值 | 0x0005 | 0x0004 | Temp_Threshold_low、Temp_Threshold_high | Operation_Status | 5.25 |
| 0xE6 | 读取温度阈值 | 0x0003 | 0x0005 | 无 | Temp_Threshold_low、Temp_Threshold_high | 5.26 |
| 0xF8 | 查询设备唯一 UID | 0x0004 | 0x0004 + UID 长度 | UID_Type | Operation_Status、UID | 5.27 |
| 0xF9 | 查询升级程序版本号 | 0x0003 | 0x0004 | 无 | Bootloader_version | 5.28 |
| 0xFA | 查询设备软件版本号 | 0x0003 | 0x0004 | 无 | Firmware_version | 5.29 |
| 0xFB | 查询设备类型 | 0x0003 | 0x0005 | 无 | Device_type | 5.30 |

| Cmd | 指令名称 | 请求 Length | 应答 / 上报 Length | Req Data | Resp Data | 详情 |
|---|---|---|---|---|---|---|
| `0xFC` | 枚举请求 / 链路通知 | 特殊帧 | 特殊帧 | 无 | 无 | 5.31 |
| `0xFD` | 心跳包 | `0x0003` | 无 | 无 | 无 | 5.32 |
| `0xFE` | 枚举 | `0x0004` | `0x0004` | `Send_num` | `Receive_num` | 5.33 |

# 5. 指令详情

本节按统一格式描述每个 Cmd 的请求数据、请求帧、应答数据、应答帧和参数枚举。

## 5.1 I2C 指定地址读 — `Cmd 0x13`

### 请求数据

Req Data 总长度：`5 byte`

请求帧 `Length = 0x0008`，低字节在前。

```
Request Frame:
  AA 55 <0x08> <0x00> <Index> <0x13> <I2c_addr> <Reg_addr_length> <Reg_addr_low>
<Reg_addr_high> <length> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
|---|---|---|
| Length | 2 byte | 固定为 `0x0008` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x13` |
| Req Data | 5 byte | `I2c_addr`、`Reg_addr_length`、`Reg_addr_low`、`Reg_addr_high`、`length` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

请求数据字段：

| 数据 | 类型 / 长度 | 说明 |
|---|---|---|
| `I2c_addr` | `uint8_t` | I2C 设备地址 |
| `Reg_addr_length` | `uint8_t` | 寄存器地址长度：`1` 表示 8 bit，`2` 表示 16 bit |
| `Reg_addr_low` | `uint8_t` | 寄存器地址低字节 |
| `Reg_addr_high` | `uint8_t` | 寄存器地址高字节 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `length` | `uint8_t` | 要读取的数据长度，最大 `64 byte` |

`Reg_addr` 按小端格式组成：

```
Reg_addr = (Reg_addr_high << 8) | Reg_addr_low
```

Chain IMU 的传感器寄存器地址为 8 bit 时，`Reg_addr_high` 通常为 `0`，`Reg_addr_length` 为 `1`。

### 应答数据

Resp Data 总长度：`1 + length byte`

应答帧 `Length = 0x0004 + length`，低字节在前。

```
Response Frame:
  AA 55 <Length_low> <Length_high> <Index> <0x13> <Operation_Status> <Data...> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | `0x0004 + length` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x13` |
| Resp Data | `1 + length byte` | `Operation_Status` 和读取数据 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

应答数据字段：

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |
| `Data` | `length byte` | 从 I2C 设备读取的数据 |

## 5.2 I2C 指定地址写 — `Cmd 0x14`

### 请求数据

Req Data 总长度：`5 + length byte`

请求帧 `Length = 0x0008 + length`，低字节在前。

```
Request Frame:
  AA 55 <Length_low> <Length_high> <Index> <0x14> <I2c_addr> <Reg_addr_length>
<Reg_addr_low> <Reg_addr_high> <length> <Data...> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | `0x0008 + length` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x14` |
| Req Data | `5 + length byte` | 地址参数、数据长度和待写入数据 |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

请求数据字段：

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `I2c_addr` | `uint8_t` | I2C 设备地址 |
| `Reg_addr_length` | `uint8_t` | 寄存器地址长度：`1` 表示 8 bit，`2` 表示 16 bit |
| `Reg_addr_low` | `uint8_t` | 寄存器地址低字节 |
| `Reg_addr_high` | `uint8_t` | 寄存器地址高字节 |
| `length` | `uint8_t` | 要写入的数据长度，最大 `64 byte` |
| `Data` | `length byte` | 写入 I2C 设备的数据 |

## 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x14> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x14` |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.3 设置 RGB 值 — `Cmd 0x20`

### 请求数据

Req Data 总长度：`2 + Num * 3 byte`

请求帧 `Length = 0x0005 + Num * 3`，低字节在前。

```
Request Frame:
  AA 55 <Length_low> <Length_high> <Index> <0x20> <RGB_Index> <Num> <R0> <G0> <B0> ...
<R(Num-1)> <G(Num-1)> <B(Num-1)> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | `0x0005 + Num * 3` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x20` |
| Req Data | `2 + Num * 3 byte` | `Index`、`Num` 和 RGB 数据 |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

请求数据字段：

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Index` | `uint8_t` | RGB 灯起始坐标，从 `0` 开始 |
| `Num` | `uint8_t` | 要设置的 RGB 灯数量；Chain IMU 只有一个 RGB 灯，通常为 `1` |
| `R` | `uint8_t` | 红色通道值，范围 `0 ~ 255` |
| `G` | `uint8_t` | 绿色通道值，范围 `0 ~ 255` |
| `B` | `uint8_t` | 蓝色通道值，范围 `0 ~ 255` |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x20> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x20` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

### 注意事项

1. `Index` 是 RGB 灯起始坐标，从 `0` 开始。
2. Chain IMU 只有一个 RGB 灯，通常使用 `Index = 0`、`Num = 1`。

## 5.4 获取 RGB 值 — `Cmd 0x21`

### 请求数据

Req Data 总长度：`2 byte`

请求帧 `Length = 0x0005`，低字节在前。

```
Request Frame:
  AA 55 <0x05> <0x00> <Index> <0x21> <RGB_Index> <Num> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Cmd | 1 byte | 固定为 `0x21` |
| Req Data | 2 byte | `RGB_Index`、`Num` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

请求数据字段：

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `RGB_Index` | `uint8_t` | RGB 灯起始坐标，从 `0` 开始 |
| `Num` | `uint8_t` | 要读取的 RGB 灯数量；Chain IMU 通常为 `1` |

## 应答数据

Resp Data 总长度：`1 + Num * 3 byte`

应答帧 `Length = 0x0004 + Num * 3`，低字节在前。

```
Response Frame:
  AA 55 <Length_low> <Length_high> <Index> <0x21> <Operation_Status> <R0> <G0> <B0> ...
<CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | `0x0004 + Num * 3` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x21` |
| Resp Data | `1 + Num * 3 byte` | 操作状态和 RGB 数据 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |
| `R` | `uint8_t` | 红色通道值，范围 `0 ~ 255` |
| `G` | `uint8_t` | 绿色通道值，范围 `0 ~ 255` |
| `B` | `uint8_t` | 蓝色通道值，范围 `0 ~ 255` |

## 5.5 设置 RGB 灯亮度 — `Cmd 0x22`

### 请求数据

Req Data 总长度：`2 byte`

请求帧 `Length = 0x0005`，低字节在前。

```
Request Frame:
  AA 55 <0x05> <0x00> <Index> <0x22> <Light> <Save_to_flash> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x22` |
| Req Data | 2 byte | `Light`、`Save_to_flash` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

请求数据字段：

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Light` | `uint8_t` | RGB 亮度，范围 `0 ~ 100`，默认值为 `40` |
| `Save_to_flash` | `uint8_t` | 只能为 `0` 或 `1`； `0` 表示不保存，`1` 表示保存 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x22> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x22` |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

### 注意事项

1. `Light` 设置成功后立即生效。
2. `Save_to_flash` 取值不是 `0` 或 `1` 时，固件返回操作失败，不改变亮度。
3. 保存至内部 Flash 需要擦除页面，频繁保存会影响 Flash 寿命；频繁调整亮度时建议使用 `Save_to_flash = 0`。

## 5.6 获取 RGB 灯亮度 — `Cmd 0x23`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x23> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x23` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x23> <Light> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x23` |
| Resp Data | 1 byte | `Light` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Light` | `uint8_t` | 当前 RGB 亮度，范围 `0 ~ 100`，默认值为 `40` |

## 5.7 读取加速度数据 — `Cmd 0x30`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x30> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x30` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`6 byte`

应答帧 `Length = 0x0009`，低字节在前。

```
Response Frame:
  AA 55 <0x09> <0x00> <Index> <0x30> <Accel_X_low> <Accel_X_high> <Accel_Y_low>
<Accel_Y_high> <Accel_Z_low> <Accel_Z_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0009` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x30` |
| Resp Data | 6 byte | 三轴 `int16_t` 加速度，按小端序排列 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_X` | `int16_t` | X 轴加速度，单位 `mg`，小端格式 |
| `Accel_Y` | `int16_t` | Y 轴加速度，单位 `mg`，小端格式 |
| `Accel_Z` | `int16_t` | Z 轴加速度，单位 `mg`，小端格式 |

### 注意事项

1. 固件读取命令执行时会更新一次 IMU 数据。
2. 单位数据来自 `g_imu_sensor_data.unit.accel_mg[3]`。
3. 当前量程对应灵敏度如下：

| 加速度量程 | 灵敏度 |
| --- | --- |
| `2G` | `0.061 mg/LSB` |
| `4G` | `0.122 mg/LSB` |
| `8G` | `0.244 mg/LSB` |
| `16G` | `0.488 mg/LSB` |

4. 单位数据由当前代码转换为 `int16_t` 后返回；正数按 `value + 0.5` 取整，负数按 `value - 0.5` 取整，超出 `int16_t` 范围时饱和。

## 5.8 读取角速度数据 — `Cmd 0x31`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x31> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x31` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

## 应答数据

Resp Data 总长度：`6 byte`

应答帧 `Length = 0x0009`，低字节在前。

```
Response Frame:
  AA 55 <0x09> <0x00> <Index> <0x31> <Gyro_X_low> <Gyro_X_high> <Gyro_Y_low> <Gyro_Y_high>
<Gyro_Z_low> <Gyro_Z_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0009` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x31` |
| Resp Data | 6 byte | 三轴 `int16_t` 角速度，按小端序排列 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_X` | `int16_t` | X 轴角速度，单位 `dps`，小端格式 |
| `Gyro_Y` | `int16_t` | Y 轴角速度，单位 `dps`，小端格式 |
| `Gyro_Z` | `int16_t` | Z 轴角速度，单位 `dps`，小端格式 |

## 注意事项

1. 固件读取命令执行时会更新一次 IMU 数据。
2. 单位数据来自 `g_imu_sensor_data.unit.gyro_mdps[3]`，返回前按 `dps = mdps / 1000` 换算。
3. 当前量程对应灵敏度如下：

| 陀螺仪量程 | 灵敏度 |
| --- | --- |
| `125 dps` | `4.375 mdps/LSB` |
| `250 dps` | `8.750 mdps/LSB` |
| `500 dps` | `17.50 mdps/LSB` |
| `1000 dps` | `35.0 mdps/LSB` |
| `2000 dps` | `70.0 mdps/LSB` |

4. 单位数据由当前代码转换为 `int16_t` 后返回；正数按 `value + 0.5` 取整，负数按 `value - 0.5` 取整，超出 `int16_t` 范围时饱和。

## 5.9 读取加速度原始值 — `Cmd 0x32`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x32> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x32` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`6 byte`

应答帧 `Length = 0x0009`，低字节在前。

```
Response Frame:
  AA 55 <0x09> <0x00> <Index> <0x32> <Accel_X_low> <Accel_X_high> <Accel_Y_low>
<Accel_Y_high> <Accel_Z_low> <Accel_Z_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0009` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x32` |
| Resp Data | 6 byte | 三轴 `int16_t` 加速度原始值，按小端序排列 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_X` | `int16_t` | X 轴加速度原始值，小端格式 |
| `Accel_Y` | `int16_t` | Y 轴加速度原始值，小端格式 |
| `Accel_Z` | `int16_t` | Z 轴加速度原始值，小端格式 |

### 注意事项

1. 原始值来自 `g_imu_sensor_data.raw.accel_raw[3]`。
2. 原始值换算为 `mg` 时，使用当前加速度量程对应灵敏度；上电默认量程为 `2G`，换算关系为 `mg = raw * 0.061`。

## 5.10 读取角速度原始值 — `Cmd 0x33`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x33> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x33` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`6 byte`

应答帧 `Length = 0x0009`，低字节在前。

```
Response Frame:
  AA 55 <0x09> <0x00> <Index> <0x33> <Gyro_X_low> <Gyro_X_high> <Gyro_Y_low> <Gyro_Y_high>
<Gyro_Z_low> <Gyro_Z_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0009` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x33` |
| Resp Data | 6 byte | 三轴 `int16_t` 角速度原始值，按小端序排列 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_X` | `int16_t` | X 轴角速度原始值，小端格式 |
| `Gyro_Y` | `int16_t` | Y 轴角速度原始值，小端格式 |
| `Gyro_Z` | `int16_t` | Z 轴角速度原始值，小端格式 |

### 注意事项

1. 原始值来自 `g_imu_sensor_data.raw.gyro_raw[3]`。
2. 原始值换算为 `mdps` 时，使用当前陀螺仪量程对应灵敏度；上电默认量程为 `250 dps`，换算关系为 `mdps = raw * 8.750`。

## 5.11 读取温度 — `Cmd 0x34`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x34> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x34` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

## 应答数据

Resp Data 总长度：`2 byte`

应答帧 `Length = 0x0005`，低字节在前。

```
Response Frame:
  AA 55 <0x05> <0x00> <Index> <0x34> <Temperature_C_x100_low> <Temperature_C_x100_high>
<CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x34` |
| Resp Data | 2 byte | `Temperature_C_x100`，小端格式 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Temperature_C_x100` | `uint16_t` | 温度摄氏值乘以 `100`，单位 `0.01 °C`，小端格式 |

## 注意事项

1. 固件读取命令执行时会更新一次 IMU 数据。
2. 当前代码返回 `imu_temperature_c_x100`。
3. 传感器原始温度换算关系为 `temperature_c = raw / 256.0 + 25.0`。

## 5.12 设置加速度量程 — `Cmd 0x35`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0x35> <Accel_Range> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x35` |
| Req Data | 1 byte | `Accel_Range` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_Range` | `uint8_t` | 加速度量程，默认值为 `2G` |

`Accel_Range` 取值：

| 值 | 名称 | 代码枚举 |
| --- | --- | --- |
| `0x00` | `2G` | `LSM6DS3TR_C_2g` |
| `0x01` | `16G` | `LSM6DS3TR_C_16g` |
| `0x02` | `4G` | `LSM6DS3TR_C_4g` |
| `0x03` | `8G` | `LSM6DS3TR_C_8g` |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x35> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x35` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.13 读取加速度量程 — `Cmd 0x36`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x36> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x36` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x36> <Accel_Range> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x36` |
| Resp Data | 1 byte | `Accel_Range` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_Range` | `uint8_t` | 当前加速度量程；取值同 `Cmd 0x35`，默认 `2G` |

## 5.14 设置陀螺仪量程 — `Cmd 0x37`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0x37> <Gyro_Range> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x37` |
| Req Data | 1 byte | `Gyro_Range` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_Range` | `uint8_t` | 陀螺仪量程，默认值为 `250 dps` |

`Gyro_Range` 取值：

| 值 | 名称 | 代码枚举 |
| --- | --- | --- |
| `0x00` | `250 dps` | `LSM6DS3TR_C_250dps` |

| 值 | 名称 | 代码枚举 |
| --- | --- | --- |
| `0x01` | `125 dps` | `LSM6DS3TR_C_125dps` |
| `0x02` | `500 dps` | `LSM6DS3TR_C_500dps` |
| `0x04` | `1000 dps` | `LSM6DS3TR_C_1000dps` |
| `0x06` | `2000 dps` | `LSM6DS3TR_C_2000dps` |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x37> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x37` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.15 读取陀螺仪量程 — `Cmd 0x38`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x38> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x38` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x38> <Gyro_Range> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x38` |
| Resp Data | 1 byte | `Gyro_Range` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_Range` | `uint8_t` | 当前陀螺仪量程；取值同 `Cmd 0x37`，默认 `250 dps` |

## 5.16 设置加速度输出频率 — `Cmd 0x39`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0x39> <Accel_ODR> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x39` |
| Req Data | 1 byte | `Accel_ODR` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_ODR` | `uint8_t` | 加速度输出频率，默认值为 `104 Hz` |

`Accel_ODR` 取值：

| 值 | 名称 | 代码枚举 |
| --- | --- | --- |
| `0x00` | `OFF` | `LSM6DS3TR_C_XL_ODR_OFF` |
| `0x01` | `12.5 Hz` | `LSM6DS3TR_C_XL_ODR_12Hz5` |
| `0x02` | `26 Hz` | `LSM6DS3TR_C_XL_ODR_26Hz` |
| `0x03` | `52 Hz` | `LSM6DS3TR_C_XL_ODR_52Hz` |
| `0x04` | `104 Hz` | `LSM6DS3TR_C_XL_ODR_104Hz` |
| `0x05` | `208 Hz` | `LSM6DS3TR_C_XL_ODR_208Hz` |
| `0x06` | `416 Hz` | `LSM6DS3TR_C_XL_ODR_416Hz` |
| `0x07` | `833 Hz` | `LSM6DS3TR_C_XL_ODR_833Hz` |
| `0x08` | `1660 Hz` | `LSM6DS3TR_C_XL_ODR_1k66Hz` |
| `0x09` | `3330 Hz` | `LSM6DS3TR_C_XL_ODR_3k33Hz` |
| `0x0A` | `6660 Hz` | `LSM6DS3TR_C_XL_ODR_6k66Hz` |
| `0x0B` | `1.6 Hz` | `LSM6DS3TR_C_XL_ODR_1Hz6` |

## 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x39> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x39` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.17 读取加速度输出频率 — `Cmd 0x3A`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x3A> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3A` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x3A> <Accel_ODR> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3A` |
| Resp Data | 1 byte | `Accel_ODR` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_ODR` | `uint8_t` | 当前加速度输出频率；取值同 `Cmd 0x39`，默认 `104 Hz` |

## 5.18 设置陀螺仪输出频率 — `Cmd 0x3B`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0x3B> <Gyro_ODR> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3B` |
| Req Data | 1 byte | `Gyro_ODR` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_ODR` | `uint8_t` | 陀螺仪输出频率，默认值为 `104 Hz` |

`Gyro_ODR` 取值：

| 值 | 名称 | 代码枚举 |
| --- | --- | --- |
| `0x00` | `OFF` | `LSM6DS3TR_C_GY_ODR_OFF` |
| `0x01` | `12.5 Hz` | `LSM6DS3TR_C_GY_ODR_12Hz5` |
| `0x02` | `26 Hz` | `LSM6DS3TR_C_GY_ODR_26Hz` |
| `0x03` | `52 Hz` | `LSM6DS3TR_C_GY_ODR_52Hz` |
| `0x04` | `104 Hz` | `LSM6DS3TR_C_GY_ODR_104Hz` |
| `0x05` | `208 Hz` | `LSM6DS3TR_C_GY_ODR_208Hz` |
| `0x06` | `416 Hz` | `LSM6DS3TR_C_GY_ODR_416Hz` |
| `0x07` | `833 Hz` | `LSM6DS3TR_C_GY_ODR_833Hz` |
| `0x08` | `1660 Hz` | `LSM6DS3TR_C_GY_ODR_1k66Hz` |
| `0x09` | `3330 Hz` | `LSM6DS3TR_C_GY_ODR_3k33Hz` |
| `0x0A` | `6660 Hz` | `LSM6DS3TR_C_GY_ODR_6k66Hz` |

## 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x3B> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3B` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.19 读取陀螺仪输出频率 — `Cmd 0x3C`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0x3C> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3C` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0x3C> <Gyro_ODR> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0x3C` |
| Resp Data | 1 byte | `Gyro_ODR` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_ODR` | `uint8_t` | 当前陀螺仪输出频率；取值同 `Cmd 0x3B`，默认 `104 Hz` |

## 5.20 IMU 事件触发 — `Cmd 0xE0`

### 请求数据

Req Data 总长度：`0 byte`

该指令由设备自动上报，主机不发送请求帧。

### 应答 / 上报数据

Resp Data 总长度：`2 byte`

主动上报帧 `Length = 0x0005`，低字节在前。

```
Event Report Frame:
  AA 55 <0x05> <0x00> <Index> <0xE0> <INT_Status> <Report_Code> <CRC> 55 AA
```

上报帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE0` |
| Resp Data | 2 byte | `INT_Status`、`Report_Code` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `INT_Status` | `uint8_t` | IMU 事件状态，多个事件可以按位同时置位 |
| `Report_Code` | `uint8_t` | 固定为 `0x10` |

`INT_Status` 位定义：

| 位 | 掩码 | 名称 | 说明 |
| --- | --- | --- | --- |
| `bit0` | `0x01` | 加速度超阈值 | `1` 表示加速度事件激活，`0` 表示解除 |
| `bit1` | `0x02` | 角速度超阈值 | `1` 表示角速度事件激活，`0` 表示解除 |
| `bit[3:2]` | `0x0C` | 温度事件 | 取值见下表 |
| `bit[7:4]` | `0xF0` | 保留 | 保留位，当前固件不使用 |

温度事件值为 `INT_Status & 0x0C`：

| 值 | 名称 | 说明 |
| --- | --- | --- |
| `0x00` | 高温告警解除 | 高温告警解除上报；RGB 红灯熄灭 |
| `0x04` | 高温告警 | 温度高于 `Temp_Threshold_high`，RGB 红灯常亮 |
| `0x08` | 低温告警 | 温度低于 `Temp_Threshold_low`，RGB 红灯常亮 |
| `0x0C` | 低温告警解除 | 低温告警解除上报；RGB 红灯熄灭 |

### 注意事项

1. 加速度、角速度和温度事件均采用连续 `5` 次采样确认机制。激活条件连续满足 `5` 次后，设备才进入告警状态并上报；解除条件连续满足 `5` 次后，设备才退出告警状态并上报。
2. 单次超过阈值或单次恢复不足以改变稳定状态，用于抑制 IMU 抖动和瞬时噪声误触发。
3. 加速度或角速度事件激活时，RGB 蓝灯常亮；解除时，RGB 蓝灯熄灭。温度事件激活时，RGB 红灯常亮；解除时，RGB 红灯熄灭。
4. 上报帧发送后，RGB 灯会在用户设置的灯效基础上叠加告警效果。

## 5.21 设置 IMU 事件触发开关 — `Cmd 0xE1`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0xE1> <Mode> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE1` |
| Req Data | 1 byte | `Mode` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

`Mode` 位定义：

| 位 | 掩码 | 取值 | 说明 |
| --- | --- | --- | --- |
| `bit0` | `0x01` | `0` / `1` | 加速度阈值触发关闭 / 打开 |
| `bit1` | `0x02` | `0` / `1` | 角速度阈值触发关闭 / 打开 |

| 位 | 掩码 | 取值 | 说明 |
| --- | --- | --- | --- |
| `bit2` | `0x04` | `0` / `1` | 温度阈值触发关闭 / 打开 |
| `bit[7:3]` | `0xF8` | `0` | 保留位必须为 `0` |

默认值：`Mode = 0x07`，即加速度、角速度和温度事件均打开。

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xE1> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE1` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.22 读取 IMU 事件触发开关 — `Cmd 0xE2`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xE2> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE2` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xE2> <Mode> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE2` |
| Resp Data | 1 byte | `Mode` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Mode` | `uint8_t` | 当前事件开关状态；位定义同 `Cmd 0xE1`，默认 `0x07` |

## 5.23 设置 IMU 六轴阈值 — `Cmd 0xE3`

### 请求数据

Req Data 总长度：`2 byte`

请求帧 `Length = 0x0005`，低字节在前。

```
Request Frame:
  AA 55 <0x05> <0x00> <Index> <0xE3> <Accel_Threshold> <Gyro_Threshold> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE3` |
| Req Data | 2 byte | `Accel_Threshold`、`Gyro_Threshold` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_Threshold` | `uint8_t` | 三轴加速度变化阈值，单位 `mg`，默认值为 `50` |
| `Gyro_Threshold` | `uint8_t` | 三轴角速度变化阈值，单位 `dps`，默认值为 `50` |

## 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xE3> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE3` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.24 读取 IMU 六轴阈值 — `Cmd 0xE4`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xE4> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE4` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`2 byte`

应答帧 `Length = 0x0005`，低字节在前。

```
Response Frame:
  AA 55 <0x05> <0x00> <Index> <0xE4> <Accel_Threshold> <Gyro_Threshold> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE4` |
| Resp Data | 2 byte | `Accel_Threshold`、`Gyro_Threshold` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Accel_Threshold` | `uint8_t` | 当前加速度阈值，默认值为 `50 mg` |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Gyro_Threshold` | `uint8_t` | 当前角速度阈值，默认值为 `50 dps` |

## 5.25 设置温度阈值 — `Cmd 0xE5`

### 请求数据

Req Data 总长度：`2 byte`

请求帧 `Length = 0x0005`，低字节在前。

```
Request Frame:
  AA 55 <0x05> <0x00> <Index> <0xE5> <Temp_Threshold_low> <Temp_Threshold_high> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE5` |
| Req Data | 2 byte | `Temp_Threshold_low`、`Temp_Threshold_high` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Temp_Threshold_low` | `uint8_t` | 低温阈值，单位 `°C`，范围 `0 ~ 255` |
| `Temp_Threshold_high` | `uint8_t` | 高温阈值，单位 `°C`，范围 `0 ~ 255` |

有效条件：`Temp_Threshold_low < Temp_Threshold_high`。默认值为低温 `10 °C`、高温 `40 °C`。

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xE5> <Operation_Status> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE5` |
| Resp Data | 1 byte | `Operation_Status` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |

## 5.26 读取温度阈值 — `Cmd 0xE6`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xE6> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE6` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`2 byte`

应答帧 `Length = 0x0005`，低字节在前。

```
Response Frame:
  AA 55 <0x05> <0x00> <Index> <0xE6> <Temp_Threshold_low> <Temp_Threshold_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xE6` |
| Resp Data | 2 byte | `Temp_Threshold_low`、`Temp_Threshold_high` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Temp_Threshold_low` | `uint8_t` | 当前低温阈值，单位 `°C` |
| `Temp_Threshold_high` | `uint8_t` | 当前高温阈值，单位 `°C` |

## 5.27 查询设备唯一 UID — `Cmd 0xF8`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前。

```
Request Frame:
  AA 55 <0x04> <0x00> <Index> <0xF8> <UID_Type> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xF8` |
| Req Data | 1 byte | `UID_Type` |
| CRC | 1 byte | `Index + Cmd + Req Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `UID_Type` | `uint8_t` | `0` 返回 4 byte UID；`1` 返回 12 byte UID；其他值无效 |

### 应答数据

Resp Data 总长度：`1 + UID 长度 byte`

有效请求的应答帧 `Length = 0x0008`（4 byte UID）或 `0x0010`（12 byte UID）。

```
Response Frame:
  UID_Type = 0: AA 55 <0x08> <0x00> <Index> <0xF8> <Operation_Status> <UID[4]> <CRC> 55 AA
  UID_Type = 1: AA 55 <0x10> <0x00> <Index> <0xF8> <Operation_Status> <UID[12]> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | `0x0008` 或 `0x0010`；无效 `UID_Type` 时为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xF8` |
| Resp Data | 1 / 5 / 13 byte | 状态码和 UID 数据 |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Operation_Status` | `uint8_t` | `0x00` 操作失败；`0x01` 操作成功 |
| `UID` | `4 / 12 byte` | `UID_Type = 0` 时返回 4 byte；`UID_Type = 1` 时返回 12 byte |

## 5.28 查询升级程序版本号 — `Cmd 0xF9`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xF9> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xF9` |
| Req Data | 0 byte | 无 |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xF9> <Bootloader_version> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xF9` |
| Resp Data | 1 byte | `Bootloader_version` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Bootloader_version` | `uint8_t` | 升级程序版本号 |

## 5.29 查询设备软件版本号 — `Cmd 0xFA`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xFA> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xFA` |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <Index> <0xFA> <Firmware_version> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xFA` |
| Resp Data | 1 byte | `Firmware_version` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Firmware_version` | `uint8_t` | 设备软件版本号；当前固件为 `0x01` |

## 5.30 查询设备类型 — `Cmd 0xFB`

### 请求数据

Req Data 总长度：`0 byte`

请求帧 `Length = 0x0003`，低字节在前。

```
Request Frame:
  AA 55 <0x03> <0x00> <Index> <0xFB> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Cmd | 1 byte | 固定为 `0xFB` |
| Req Data | 0 byte | 无 |
| CRC | 1 byte | `Index + Cmd` 累加和低 8 位 |

### 应答数据

Resp Data 总长度：`2 byte`

应答帧 `Length = 0x0005`，低字节在前。

```
Response Frame:
  AA 55 <0x05> <0x00> <Index> <0xFB> <Device_type_low> <Device_type_high> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0005` |
| Index | 1 byte | 设备在 Chain 链路中的位置 |
| Cmd | 1 byte | 固定为 `0xFB` |
| Resp Data | 2 byte | `Device_type_low`、`Device_type_high` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Device_type_low` | `uint8_t` | 设备类型低字节 |
| `Device_type_high` | `uint8_t` | 设备类型高字节 |

`Device_type` 按小端格式组成：

```
Device_type = (Device_type_high << 8) | Device_type_low
```

Chain IMU 当前设备类型码为 `0x0010`。

## 5.31 枚举请求 / 链路通知 — `Cmd 0xFC`

### 请求数据

该 Cmd 为 Chain 内部特殊帧，不作为普通主机请求帧发送。

### 通知帧

Resp Data 总长度：`0 byte`

固件使用的通知帧格式如下：

```
Enumeration Request Frame:
  AA 55 <0x03> <0x00> <0xFF> <0xFC> <CRC> 55 AA
```

字段说明：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 特殊帧使用 `0xFF` |
| Cmd | 1 byte | 固定为 `0xFC` |
| Data | 0 byte | 无 |
| CRC | 1 byte | `0xFF + 0xFC = 0xFB` |

### 注意事项

1. 该帧用于 Chain 链路枚举请求和链路状态通知。
2. 固件对该特殊帧按链路逻辑转发，不进入普通命令处理流程。
3. 转发时保持特殊帧格式不变。

## 5.32 心跳包 — `Cmd 0xFD`

### 心跳帧

Req Data 总长度：`0 byte`

心跳帧 `Length = 0x0003`，低字节在前；心跳帧使用 `Index = 0xFF`。

```
Heartbeat Frame:
  AA 55 <0x03> <0x00> <0xFF> <0xFD> <CRC> 55 AA
```

字段说明：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0003` |
| Index | 1 byte | 固定为 `0xFF` |
| Cmd | 1 byte | 固定为 `0xFD` |
| Data | 0 byte | 无 |

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| CRC | 1 byte | `0xFF + 0xFD = 0xFC` |

### 注意事项

1. 心跳用于 Chain 设备之间的定时通信和末端设备判断。
2. 主机可以通过心跳状态判断 Chain 设备是否连接。
3. 心跳请求不需要主机响应，设备不会为该心跳帧生成普通命令应答帧。

## 5.33 枚举 — `Cmd 0xFE`

### 请求数据

Req Data 总长度：`1 byte`

请求帧 `Length = 0x0004`，低字节在前；枚举帧使用 `Index = 0xFF`。

```
Request Frame:
  AA 55 <0x04> <0x00> <0xFF> <0xFE> <Send_num> <CRC> 55 AA
```

请求帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 特殊帧使用 `0xFF` |
| Cmd | 1 byte | 固定为 `0xFE` |
| Req Data | 1 byte | `Send_num` |
| CRC | 1 byte | `Index + Cmd + Send_num` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Send_num` | `uint8_t` | 当前枚举过程中已记录的设备数量；初始值通常为 `0` |

### 应答数据

Resp Data 总长度：`1 byte`

应答帧 `Length = 0x0004`，低字节在前。

```
Response Frame:
  AA 55 <0x04> <0x00> <0xFF> <0xFE> <Receive_num> <CRC> 55 AA
```

应答帧字段：

| 字段 | 长度 | 说明 |
| --- | --- | --- |
| Length | 2 byte | 固定为 `0x0004` |
| Index | 1 byte | 特殊帧使用 `0xFF` |
| Cmd | 1 byte | 固定为 `0xFE` |
| Resp Data | 1 byte | `Receive_num` |
| CRC | 1 byte | `Index + Cmd + Resp Data` 累加和低 8 位 |

| 数据 | 类型 / 长度 | 说明 |
| --- | --- | --- |
| `Receive_num` | `uint8_t` | Chain 链路中的设备数量 |

---

# 6. 状态码 / 错误码

## 6.1 Operation_Status

`Operation_Status` 用于表示命令执行结果。具体使用该状态码的指令，会在对应的应答帧中返回 1 byte `Operation_Status`。

| 值 | 名称 | 说明 |
| --- | --- | --- |
| `0x00` | 操作失败 | 命令参数无效、硬件操作失败或当前操作未成功完成 |
| `0x01` | 操作成功 | 命令已成功执行 |

---

# 7. 版本记录

| 文档版本 | 固件版本 | 日期 | 说明 |
| --- | --- | --- | --- |
| `V1.1` | `0x01` | 2026-08-20 | 初始版本 |